

Smart Contract Audit Report

Contract: sample_contract

Date: 2025-07-08 23:00:40

Risk Summary

Risk Score: 30/100

Risk Level: Medium

Vulnerabilities Found: 3

Vulnerability Findings

1. Unchecked arithmetic operation

Severity: Unknown

Location: balance += amount

Description: Operation 'balance += amount' might cause overflow/underflow. Consider using safe math checks.

2. Unchecked arithmetic operation

Severity: Unknown

Location: balance -= amount

Description: Operation 'balance -= amount' might cause overflow/underflow. Consider using safe math checks.

3. Unchecked arithmetic operation

Severity: Unknown

Location: balance -= amount

Description: Operation 'balance -= amount' might cause overflow/underflow. Consider using safe math checks.

AI-Powered Analysis

****Audit Summary for sample_contract****

The audit of the `sample\_contract` has identified three instances of unchecked arithmetic operations, which can lead to potential overflow/underflow vulnerabilities. Below is a detailed analysis of each vulnerability, including severity, impact, potential exploit paths, and mitigation strategies.

****Vulnerability 1: Unchecked arithmetic operation in `balance += amount`****

* **Severity:** Medium

* **Impact:** An attacker can manipulate the `balance` variable to exceed the maximum value it can hold, causing an overflow. This can result in unexpected behavior, such as an attacker being able to withdraw more funds than they are entitled to.

* **Potential Exploit Path:** An attacker can call the contract function that increments the `balance` variable with a large `amount` value, causing an overflow.

* **Mitigation Strategy:** Use safe math libraries, such as OpenZeppelin's `SafeMath`, to perform arithmetic operations. These libraries include built-in checks to prevent overflows and underflows. For example, `balance = balance.add(amount)` can be replaced with `balance = SafeMath.add(balance, amount)`.

Vulnerability 2 and 3: Unchecked arithmetic operation in `balance -= amount`

* **Severity:** Medium

* **Impact:** An attacker can manipulate the `balance` variable to decrease below zero, causing an underflow. This can result in unexpected behavior, such as an attacker being able to withdraw more funds than they have deposited.

* **Potential Exploit Path:** An attacker can call the contract function that decrements the `balance` variable with a large `amount` value, causing an underflow.

* **Mitigation Strategy:** Similar to Vulnerability 1, use safe math libraries to perform arithmetic operations. For example, `balance -= amount` can be replaced with `balance = SafeMath.sub(balance, amount)`.

General Advice for Developers:

1. **Use safe math libraries:** When performing arithmetic operations, always use libraries that include built-in checks for overflows and underflows.
2. **Test for edge cases:** Ensure that your contract is tested for edge cases, such as very large or very small input values, to identify potential vulnerabilities.
3. **Follow best practices:** Follow established best practices for smart contract development, such as those outlined in the Smart Contract Security Best Practices guide.
4. **Perform regular security audits:** Regularly audit your contracts to identify and address potential vulnerabilities before they can be exploited.

By addressing these vulnerabilities and following best practices, you can significantly reduce the risk of your contract being exploited and ensure the security and integrity of your users' funds.

Generated by DeFi Sentinel